Exp5 - weka
Exp6 - apriori

```python
import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules

df = pd.DataFrame({
    'Milk': [1, 1, 0, 1, 1],
    'Bread': [1, 1, 1, 1, 0],
    'Butter': [0, 1, 1, 1, 1],
    'Eggs': [1, 0, 1, 1, 0]
})

print("Dataset:")
print(df)

frequent = apriori(df, min_support=0.4, use_colnames=True)

print("\nFrequent Itemsets:")
print(frequent)

rules = association_rules(frequent, metric="confidence", min_threshold=0.6)

print("\nAssociation Rules:")
print(rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']])
```

Exp4 - weka + python

```python

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv('filepath')

print("Original Data:")
print(df)

print("\nFirst 10 Rows:")
print(df.head(10))

print("\nStatistical Description:")
print(df.describe())

print("\nMissing Values:")
```

```python
print(df.isnull().sum())

df['Name'] = df['Name'].fillna('Unknown')
df['Marks'] = df['Marks'].fillna(df['Marks'].mean())

print("\nCleaned Data:")
print(df)

sns.histplot(df['Marks'], bins=5, kde=True)
plt.title('Histogram of Marks')
plt.xlabel('Marks')
plt.ylabel('Frequency')
plt.show()

plt.pie(
    df['Name'].value_counts(),
    autopct='%1.1f%%'
)

plt.title('Pie Chart of Students by Name')
plt.show()

sns.boxplot(x=df['Marks'])
plt.title('Boxplot of Marks')
plt.show()
```

Exp 7

```python
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import classification_report, confusion_matrix


df = pd.read_csv('/content/sample_data/EEG Eye State.csv')

X = df.drop('eyeDetection', axis=1)
y=df['eyeDetection']

scalar = StandardScaler()
X_scaled = scalar.fit_transform(X)
```

```
kmeans = KMeans(n_clusters=2, random_state=42, n_init=1)
clusters = kmeans.fit_predict(X_scaled)

mapped_cluster = np.where(
    clusters==0,
    1 if np.mean(y[clusters==0]) >0.5 else 0,
    1 if np.mean(y[clusters==1]) >0.5 else 0
)

print(classification_report(y, mapped_cluster))
print(confusion_matrix(y, mapped_cluster))
```

Exp 8

Python

```python
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import dendrogram, linkage

# Dataset
data = pd.read_csv("./hierarchical_data.csv")

# Select numeric columns
X = data[["Age", "Marks"]]

# Create dendrogram
linked = linkage(X, method="ward")

dendrogram(linked)
plt.title("Dendrogram")
plt.xlabel("Data Points")
plt.ylabel("Distance")
plt.show()

# Hierarchical clustering
model = AgglomerativeClustering(n_clusters=3, linkage="ward")

labels = model.fit_predict(X)

# Display clusters
data["Cluster"] = labels
print(data)

# Visualize clusters
plt.scatter(X["Age"], X["Marks"], c=labels)
plt.xlabel("Age")
plt.ylabel("Marks")
plt.title("Hierarchical Clustering")
plt.show()
```

Exp 9

Python

```python
import pandas as pd

df = pd.read_csv("./weather.csv")
total = len(df)


yes = len(df[df["PlayTennis"] == "Yes"])
no = len(df[df["PlayTennis"] == "No"])

p_yes = yes / total
p_no = no / total

# test data
values = ["Sunny", "Cool", "High", "Strong"]
attribs = ["Outlook", "Temperature", "Humidity", "Wind"]

for i in range(len(attribs)):
    attrib = attribs[i]
    value = values[i]

    count_yes = len(df[(df[attrib] == value) & (df["PlayTennis"] == "Yes")])
    count_no = len(df[(df[attrib] == value) & (df["PlayTennis"] == "No")])

    p_yes = p_yes * count_yes / yes
    p_no = p_no * count_no / no

print("yes prob", p_yes)
print("no prob", p_no)

if p_yes > p_no:
    print("Play Tennis: Yes")
else:
    print("Play Tennis: No")
```

Exp 10 - page hits

```python
n=3
ite = 10

graph = [
    [0, 1, 1],
    [0, 0, 1],
    [1, 0, 0]
]

auth = [1]*n
hub = [1]*n

for i in range(ite):

    for j in range(n):
        auth[j]=0
        for k in range(n):
            auth[j] += hub[k] * graph[k][j]

    for j in range(n):
        hub[j]=0
        for k in range(n):
            hub[j] += auth[k] * graph[j][k]

    a = sum(auth)
    h = sum(hub)

    auth = [x/a for x in auth]
    hub = [x/h for x in hub]

    print('iteration',i+1)
    print('Auhtority score', auth)
    print('hub score', hub)

print('Final ')
print('Auhtority score', auth)
print('hub score', hub)
```

Exp 10 - page rank

Python

```
n=3
d=0.85
ite=10

graph = [
    [0, 1, 1],
    [0, 0, 1],
    [1, 0, 0]
]

rank =[1/n, 1/n, 1/n]

for i in range(ite):
    new_rank=[]
    for j in range(n):
        total = 0
        for k in range(n):
            if graph[k][j] == 1:
                total += rank[k] / sum(graph[k])

        pr = (1-d)/n + d*total
        new_rank.append(pr)

    print(new_rank)
    rank=new_rank

print("\n Final rank is", rank)
```

Python

```
#without damping
n = 3
ite = 10

graph = [
    [0, 1, 1],
    [0, 0, 1],
    [1, 0, 0]
]

rank = [1/n, 1/n, 1/n]

for i in range(ite):
    new_rank = []

    for j in range(n):
```

```
        total = 0

        for k in range(n):
            if graph[k][j] == 1:
                total += rank[k] / sum(graph[k])

        new_rank.append(total)

    print(new_rank)
    rank = new_rank

print("\nFinal rank is", rank)
```

Sql

```
SQL
### 🟦SQL Database Basic Structure

```treeview
Database[Company]
    ├── Table[employess]
    │       ├── Row (record)
    │       ├── Row
    │       └── Row
    │
    ├── Table[department]
    │       ├── Row
    │       └── Row
    │
    └── Table[project]

Database
+── ── ── ── ── ── ── ── ── ── ── ──+
|   +── ── ── ──+   +── ── ── ──+   |
|   | employees |   | department|   |
|   +── ── ── ──+   +── ── ── ──+   |
|   | id  name  |   | id  name  |   |
|   | 1    A    |   | 10   HR   |   |
|   | 2    B    |   | 20   IT   |   |
|   +── ── ── ──+   +── ── ── ──+   |
+── ── ── ── ── ── ── ── ── ── ──+
```

> row → tuple table → entity
```

⚙️Database Query

```sql
Database Queries
    ├── Creating database
    |       ├── CREATE DATABASE db_name;
    |       └── CREATE DATABASE IF NOT EXISTS db_name; → #for handling error
    |
    ├── Deleting database
    |       ├── DROP DATABASE db_name;
    |       └── DROP DATABASE IF EXISTS db_name; → #for handling error
    |
    ├── SHOW DATABASES; → #to show all existing database
    ├── USE db_name; → #enter into a particular database to perform queries on
table of db_name database
    └── SHOW TABLES; → #to show all table in a database
```

> -- → single line comment
> /* */ → multiline comment

⚙️Table Query

```sql
Table Queries
    ├── CREATE → #schema/columns
    ├── INSERT
    ├── UPDATE
    ├── ALTER
    ├── TRUNCATE
    └── DELETE
```

⚙️Creating Table Query

```sql
CREATE TABLE table_name(
    column_name1 datatype constraint,
    column_name2 datatype constraint,
)
```

```treeview
datatype
  ├── char(val) → char(5)
  ├── *varchar(val) → varchar(10)
  ├── blob(val) → blob(1000)
  ├── *int
  ├── tinyint
  ├── bigint
  ├── bit(val) → range from 1 to 64 → bit(2) → 00 01 10 11
  ├── *float
  ├── *double
  ├── boolean
  ├── *date
  └── year
```

🧠 Constraints

```sql
Constraints → #means restriction or limit
    ├── PRIMARY KEY
    ├── FOREIGN KEY
    ├── NOT NULL
    │     ├── #NULL → #not allowed ❌
    │     ├── #'' → #allowed ✅
    │     └── #' ' → #allowed ✅
    │
    ├── UNIQUE → #all values in column are different
    ├── DEFAULT value → # sets the default value of a column
    └── CHECK → #apply condition on columns
```

```sql
email INT NOT NULL UNIQUE
salary INT DEFAULT 25000
age INT CHECK (age>=18 AND age<=50)
```

> What are **Keys**?
>
> - Keys are special columns in the table

> **Primary Key:**
>
> - It is a column (or set of columns) in a table that uniquely identifies each row. (a unique id)
>

> - There is only 1PK & it should be `NOT NULL`
>

```sql
cols_name datatype PRIMARY KEY

CREATE TABLE users{
    name VARCHAR(50)
    age INT CHECK (age>=18),
    id INT PRIMARY KEY,
    email VARCHAR(50) NOT NULL UNIQUE,
    followers INT DEFUALT 0,
    following INT DEFUALT 0
};
```

> **Foreign Key:**
>
> - A foreign key is a column (or set of columns) in a table that refers to the primary key in a table
>
> - FKs can have duplicate & null values.
>
> - There can be multiple FKs.
>
> - FK can take null as well as value but only those value which exist in foreign table column consider as foreign key
>

```sql
FOREIGN KEY (cols_name in current table) REFERENCES table_name(cols_name)

CREATE TABLE posts (
    id INT PRIMARY KEY,
    content VARCHAR(100),
    user_id INT,
    user_email VARCHAR(50),
    FOREIGN KEY (user_id) REFERENCES users(id),
    FOREIGN KEY (user_email) REFERENCES users(email),
);
```

```mermaid
erDiagram

    user ||--o{  post: user_id
    user ||--o{  post: user_email
```

```
    user {
        int id
        int age
        string name
        string email
        int followers
        int following
    }

    post {
        int id
        string content
        int user_id
        string user_email
    }
```

> to see ER diagram in mysql workbench Database → Reverse Engineering

---

⚙️ INSERT Query

> insert data new data into table

```sql
INSERT INTO table_name (colname1, colname2)
VALUES
(col1_v1, col2_v1),
(col2_v2, col2_v2);
```

⚙️ SELECT Query

> Selects & Show data from the DB

```sql
SELECT cols1, cols2, cols3... FROM table_name → #show particular column from
table;
```

```sql
SELECT * FROM table_name → #show all column from table;
```

🧠 Cluase

- **WHERE** Clause : To define some conditions on rows.

  ```sql
  SELECT * FROM table_name WHERE (condition);
  ```

  ```treeview
  operator used in where clause
      ├── Arithmetic Operators : +(addition) , -(subtraction), *(multiplication), /(division), %(modulus)
      ├── Comparison Operators : = (equal to), != (not equal to), >, >=,<,<=
      ├── Logical Operators : AND, OR, NOT, IN, BETWEEN, ALL, LIKE, ANY
      └── Bitwise Operators : & (Bitwise AND), | (Bitwise OR)
  ```

  ```sql
  SELECT * FROM users WHERE age+1 =18
  SELECT name,age,email FROM user WHERE age>=30

  SELECT * FROM users WHERE age>=30 AND age<=50

  SELECT * FROM users WHERE age BETWEEN 30 AND 50 → #show those row which lie between 30 and 50
  SELECT name, age FROM users WHERE age NOT BETWEEN 30 AND 50 → #show those row which not lie between 30 and 50

  SELECT name, age FROM users WHERE age IN (14, 16) → #show only those row which have age 14 and 16, not between 14 to 16
  SELECT name, age,email FROM users WHERE email IN ('bob@gmail.com', 'modi@gmail.com') ↑ #same as above
  ```

  > **`IN`** clause require tuple(list)

- **LIMIT** Clause - Sets an upper limit on number of rows(tuples) to be returned

  ```sql
  SELECT col7, col2 FROM table_name LIMIT val; → #general case

  SELECT * FROM users LIMIT 5 → #only 5 row are shown;
  SELECT * FROM users WHERE age>=15 LIMIT 2;
  ```

- **ORDER BY** Clause - To sort in **ascending (`ASC`)** or **descending order (`DESC`)**

  ```sql
  SELECT col7, col2 FROM table_name ORDER BY col_name ASC/DESC; → #general case

  SELECT * FROM users ORDER BY age ASC;
  SELECT * FROM users ORDER BY age ASC LIMIT 2;
  ```

- **Aggregate Functions** - Aggregate functions perform a calculation on a set of values, and return a single value.

  ```treeview
  Aggregate Functions
      ├── COUNT()
      ├── MAX()
      ├── MIN()
      ├── SUM()
      └── AVG()
  ```

  ```sql
  select count(*) from user;
  select count(id) from user where age<20;

  select max(age) from user where age<20;
  select min(followers) from user where name in ('adam', 'eve', 'adameve');
  select max(followers) from user where name in ('adam', 'eve', 'adameve');

  select sum(age) from user;
  select avg(age) from user;
  ```

- **GROUP BY** Clause:

  - Groups rows that have the same values into summary rows.

  - It collects data from multiple records and groups the result by one or more column.

  - Generally we use `GROUP BY` with some aggregation function.


  ```sql
  SELECT col1, Aggregate_Func()... FROM table_name GROUP BY cols1(s); → #general

  SELECT age, count(age), sum(followers), min(followers), max(followers) FROM user GROUP BY age;
```

```sql
  SELECT age, count(age), sum(followers), min(followers), max(followers)
  FROM user
  GROUP BY age
  ORDER BY count(age) ASC;
  ```

- **HAVING** Clause:

  - Similar to `WHERE`(applies some condition on rows),

  - It is used when we want to apply any condition after grouping.

  - `WHERE` is for the table, `HAVING` is for a group

  - Grouping is necessary for `HAVING`


  ```sql
  SELECT col1, Aggregate_Func()... FROM table_name GROUP BY cols1(s) HAVING
(condition); → #general

  SELECT age, count(age), sum(followers), min(followers), max(followers)
  FROM user
  GROUP BY age
  HAVING age>=18 AND age<=30
  ORDER BY count(age) ASC;
  ```

- General order:

  ```sql
  SELECT column(s), Aggregate_Func()...
  FROM table_name
  WHERE (condition)
  GROUP BY column(s)
  HAVING (condition)
  ORDER BY column(s) ASC;

  SELECT age, sum(followers) , min(followers), max(followers)
  FROM user
  WHERE age IN (14,15,16,17,18,19)
  GROUP BY age
  HAVING min(followers)>250
  ORDER BY age ASC;
  ```

⚙️UPDATE Query

> To update existing rows

```sql
UPDATE table_name
SET cols1=val, cols2=val
WHERE condition

UPDATE user
set age=51, follower=2500
WHERE name='modi'

UPDATE user
set age=age+1, follower=2500 → #+= ❌ SQL
WHERE name='trump'
```

⚙️DELETE Query

> To delete existing rows

```sql
DELETE FROM table_name WHERE (condition); → #where clause is imp otherwise
whole row will be deleted
```

⚙️ALTER Query

> To change the schema by changing or modify the column

```treeview
ALTER TABLE table_name
    ├── ADD COLUMN → to add new column in existing table
    ├── DROP COLUMN → to remove existing column from table
    ├── CHANGE COLUMN
    |       ├── to change the name of column
    |       └── only name, datatype and constraint remain same
    ├── MODIFY → to change datatype and constraint of column
    └── RENAME TO → to rename a table
```

- ADD COLUMN

  ```sql
  ALTER TABLE table_name
  ADD COLUMN column_name datatype constraint;

  ALTER TABLE users
  ADD COLUMN youtube_link varchar(100) DEFAULT NULL;
  ```

- DROP COLUMN

  ```sql
  ALTER TABLE TABLE_NAME
  DROP COLUMN  column_name;

  ALTER TABLE USERS
  DROP COLUMN youtube_link;
  ```

- CHANGE COLUMN

  ```sql
  ALTER TABLE TABLE_NAME
  CHANGE COLUMN old_name new_name datatype constraint;

  ALTER TABLE USERS
  CHANGE COLUMN youtube_link yt_link varchar(100) DEFAULT NULL;
  ```

- MODIFY

  ```sql
  ALTER TABLE TABLE_NAME
  MODIFY cols_name new_datatype new_constraint;

  ALTER TABLE USERS
  MODIFY yt_link char(80) NOT NULL;
  ```

- RENAME TO

  ```sql
  ALTER TABLE table_name
  RENAME TO new_table_name;

  ALTER TABLE USERS
  RENAME TO usersInfo
  ```

⚙️TRUNCATE Query

> 👉 Truncate (to delete table's data)
> 👉 delete whole rows

```sql
TRUNCATE TABLE table_name ;
```

⚙️ SQL With Node

> install npm package called mysql2

```bash
npm install mysql2
```

```js
const mysql = require('mysql2');
const connection = mysql.createConnection({
    host: "localhost",
    user: "root",
    password: "yourpassword",
    database: "db_name"
})
connection.query(query,[arr], (err, result)=>if(!err) {console.log(result)})
connection.end() //to end the connection
```

```js
//using pool → recommended for production
import mysql from "mysql2/promise";
const pool = mysql.createPool({
    host: "localhost",
    user: "root",
    password: "dbpwd",
    database: "dbname",
    waitForConnections: true,
    connectionLimit: 10,
    queueLimit: 0
});
```

```
await pool.getConnection()
    .then(() => console.log("sql connected"))
    .catch(() => console.log("sql not connect"))

const q = 'select * from users';
const [result, err] = await pool.execute(q);
console.log(result);
pool.end();
```

```sql
#insert using placeholder
const q = "INSERT INTO users(id, username, email, pwd) values(?,?,?)";
const user = ["123", "123venkates", "venkates@gmail.com", "123@venkates"];
connection.execute(q, user, (err, result)=>{console.log(result)})

#insert bulk data using placeholder
const q = "INSERT INTO users(id, username, email, pwd) values ?";
const users = [
    ["123a", "123venkatesa", "venkatesa@gmail.com", "123@venkatesa"],
    ["123b", "123venkatesb", "venkatesb@gmail.com", "123@venkatesb"],
    ["123c", "123venkatesc", "venkatesc@gmail.com", "123@venkatesc"]
]
connection.query(q, [users], (err, result)=>{console.log(result)})
```

Example code

```js
import mysql from "mysql2/promise";
import express from "express";

const pool = mysql.createPool({
    host: "localhost",
    user: "root",
    password: "Vyoma@0915",
    database: "test",
    waitForConnections: true,
    connectionLimit: 10,
    queueLimit: 0
});

await pool.getConnection()
    .then(() => console.log("sql connected"))
    .catch(() => console.log("sql not connect"))

const app = express();
```

```
app.get("/", async (req, res) => {
    const q = 'select * from users';
    const [result, err] = await pool.execute(q);
    console.log(result);
    res.json(result)
})

const PORT = 8000;
app.listen(PORT, () => {
    console.log(`server has started at PORT: ${PORT}`);
})
```

> not need to end connection
